

**ALGORITHMS AND DATA STRUCTURES
(25COP923)**

Sample Practice Exam Paper

2 hours

Please fill in:

ID number:

Desk number:

This examination is to take place in-person at a central University venue under exam conditions. The standard length of time for this paper is **2 hours**.

You will not be able to leave the exam hall for the first 30 or final 15 minutes of your exam.

Your invigilator will collect your exam paper when you have finished.

Help during the exam

Invigilators are not able to answer queries about the content of your exam paper. Instead, please make a note of your query in your answer script to be considered during the marking process.

If you feel unwell, please raise your hand so that an invigilator can assist you.

You may **not** use a calculator for this exam.

Write your answer for every question in the appropriate space. Indicate within that space if you give additional parts of your answer elsewhere.

概念解释

1. Definitions. Provide informal definitions of the concepts below. [10]

- (a) What is a **graph**? Provide an informal definition and explain the difference between **directed** and **undirected** graphs. [5]

A **graph** G is a pair (V, E) where
 V is a set of vertices
 or E is a set of edges connecting pairs of vertices

In an undirected graph, edges have no direction and represent a two-way connection between vertices.

In a directed graph, edges have a direction, meaning each edge goes from one vertex to another specified vertex.

Tick here if you continue at the end of the booklet: ☐

- (b) What is an **hard problem**? Explain the difference between P and NP problems. [5]

A problem is NP-Hard if a polynomial-time algorithm solving it would refute the P = NP conjecture

Tick here if you continue at the end of the booklet: ☐

2. Algorithm analysis.

[15]

What is the worst-case time complexity of the following algorithm in big-Oh notation? What task is performed by the algorithm? How is that task achieved? (Write the respective answers into the boxes.)

Procedure(A, n)

```

1  for  $i \leftarrow 1$  to  $n - 1$  do
2      for  $j \leftarrow i + 1$  to  $n$ 
3          if  $A[i] = A[j]$  then
4              return true
5  return false

```

Complexity?	$O(n^2)$
Task performed? 算法目的是什么	the algorithm checks whether the array contains any duplicate elements
How achieved?	The algorithm uses two nested loops to compare all pairs of elements in the array. If any two elements are equal, it returns true; otherwise, it returns false after all comparisons.

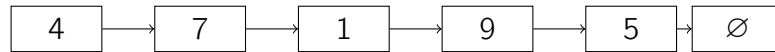
Tick here if you continue at the end of the booklet: ☐

数据结构

3. Data structures.

[20]

(a) Consider the singly linked list shown below:



(i) Fill out the following table.

[8]

Traversal遍历	Sequence of nodes visited访问结点的顺序
Sequence of nodes visited in a head-to-tail traversal	4 7 1 9 5
Sequence of nodes visited in a search for value 9	4 7 1 9
Number of elements in the list	5
Value of the head of the list	4
Value of the tail of the list	5

Tick here if you continue at the end of the booklet: ☐

(b) Suppose the linked list data structure is implemented using nodes containing:

- a data field (`value`)
- a pointer to the next node (`next`)

Write the pseudocode of the recursive primitives 递归原语 for the following linked list operations.

Use the following function signatures:

[12]

(i) **Forward Traversal** (`Head` $\rightarrow$ `Tail`)

`Traverse(node)`

Tick here if you continue at the end of the booklet: ☐

(ii) **Search for a value**

`Search(node, target)`

```
x = L.head
while x ≠ NIL and x.key ≠ k
    x = x.next
return x
```

1. Not Recursive: The code in the image uses a while loop to traverse the linked list. The question explicitly asks for "recursive primitives," which means the function must call itself (recursion) rather than using a loop (iteration).

2. Variable Name Mismatch:

The function signature provided is `Search(node, target)`.

However, the code body uses `L.head` (ignoring the `node` parameter), `x.key` (whereas the question specifies a `value` field), and `k` (instead of the parameter `target`).

Tick here if you continue at the end of the booklet: ☐

(iii) **Count the nodes in the list**

CountNodes(*node*)

Tick here if you continue at the end of the booklet: ☐

图算法

4. Graphs. [35]

A large logistics company is planning an upgrade of its delivery drone network. The company operates many one-way flight routes between distribution hubs. Each route has an associated travel time that depends on prevailing winds.

To guarantee fast deliveries, management wants to mark certain routes as *priority air corridors*. Drones travelling along a priority corridor experience reduced delays, making these routes effectively “fast lanes”.

The company imposes the following rules:

- Each existing flight route must remain available, either as a normal route or as a faster priority corridor.
- Priority corridors follow the direction of the underlying flight route.
- For a designated central hub s , every other hub must be reachable from s using only priority corridors.
- The total travel time of all priority corridors must be as small as possible.

The aim is to build the minimal set of directed priority routes that guarantees fast reachability from the single source hub s .

- (a) Explain how this problem can be described and formalised in terms of graphs. [10]

Single-Source Shortest Path

Tick here if you continue at the end of the booklet: ☐

- (b) Design an algorithm that determines which routes should be upgraded to priority air corridors. Describe it in words and provide high-level pseudocode. State which known algorithm your solution is based on. [15]

Dijkstra's Algorithm

Dijkstra($G = (V, E), w, s$)

1 $C \leftarrow \{s\}$

2 $\ell(s) \leftarrow 0; \ell(v) \leftarrow \infty$ for all vertices

$v \in V \setminus \{s\}$

3 predecessor(s) $\leftarrow$ NIL

4 while $C \neq V$ do

5 $(u, v) \leftarrow$ edge minimising $\ell(u) +$

$w_{uv}, u \in C, v \in V \setminus C$

6 Add v to C

7 $\ell(v) \leftarrow \ell(u) + w_{uv}$

8 predecessor(v) $\leftarrow u$

9 return ℓ , predecessor

Tick here if you continue at the end of the booklet: ☐

(c) Discuss a suitable representation of the directed graph and an implementation of your algorithm. Mention any auxiliary辅助的 data structures and estimate the running time of your approach. [10]

```

Bellman-Ford( $G = (V, E), w, s$ )
1  $A[0][s] \leftarrow 0; A[0][v] \leftarrow \infty$  for all vertices  $v \in V \setminus \{s\}$ 
2 for  $i \leftarrow 1$  to  $|V| - 1$  do
3   for each  $v \in V$  do
4      $A[i][v] \leftarrow \min\{A[i-1][v], \min_{(u,v) \in E} \{A[i-1][u] + w_{uv}\}\}$ 
5 return  $A$ 

```

$O(|V||E|)$

Tick here if you continue at the end of the booklet: ☐

5. **Kruskal's** Algorithm.

[20]

(a) Let $G = (V, E)$ be a graph and w a function assigning a weight to each edge of G .

Define:

- what is a *spanning tree* of G ;
- what is a *minimum spanning tree* (MST) of G .

[5]

Minimum Spanning Trees

Goal

Connect all vertices with the minimum total cost

Applications include designing efficient networks

The Problem

Input An undirected graph $G = (V, E)$

with cost c_e for each edge $e \in E$

Output Minimum spanning tree T that spans all vertices

Tick here if you continue at the end of the booklet: ☐

(b) In the following questions refer to the Kruskal's algorithm.

(i) Briefly state the problem solved by Kruskal's algorithm, indicating:

- the input to the algorithm;
- the output of the algorithm.

[5]

Tick here if you continue at the end of the booklet: ☐

(ii) Here is a simple, non-optimised version of Kruskal's algorithm:

Kruskal-simple($G = (V, E), w$)

1 sort edges in E by non-decreasing weight

2 $MST \leftarrow \emptyset$

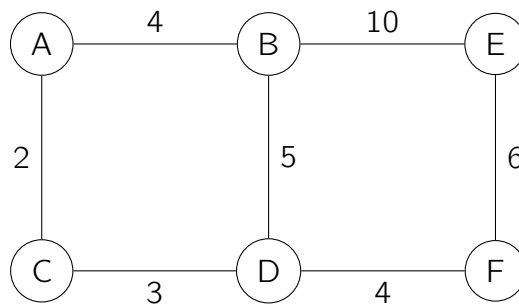
3 **for each** edge (u, v) in sorted order **do**

4 **if** adding (u, v) to MST does not create a cycle **then**

5 add (u, v) to MST

6 **return** MST

Consider the following graph:



Run the simple Kruskal algorithm on this graph. Complete the table below after each step, indicating:

- the edge currently being considered (sorted by weight);
- whether the algorithm chooses it;
- the current set of edges in the MST.

[10]

Step	Edge	Taken?	Current MST edges
1	(A,C)	2	(A,C)
2	(C,D)	5	(A,C)(C,D)
3	(D,F)	4	(A,C)(C,D)(D,F)
4	F,E	6	(A,C)(C,D)(D,F)(F,E)
5	E,B	10	(A,C)(C,D)(D,F)(F,E)(E,B)
6			
7			